

Forward Process (Diffusion)

- Corrupting the data x_0 with noise over a sequence of T time steps.
- A Markov chain where each step slightly degrades the data by adding Gaussian noise.

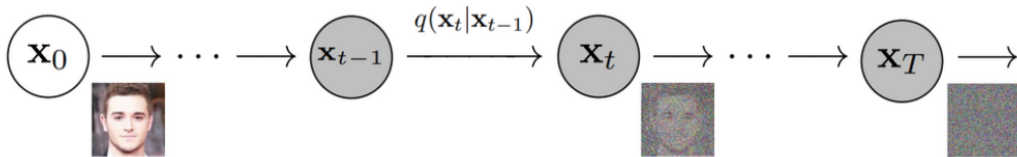


Figure: Forward Process.

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I). \quad (1)$$

Forward Process (Diffusion)

Sampling x_t from conditional distribution $q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$,

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon. \quad (2)$$

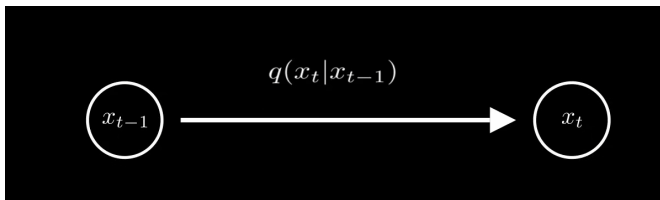
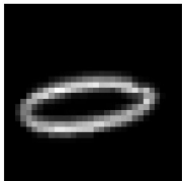


Figure: Forward Process (Markov chain).

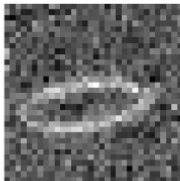
- where $\epsilon \sim \mathcal{N}(0, I)$ is Gaussian noise.
- β_t is the noise schedule that controls the amount of noise added at each step.

Forward Process (Diffusion)

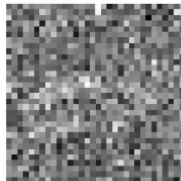
t = 0



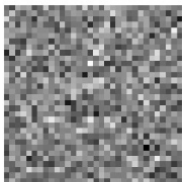
t = 100



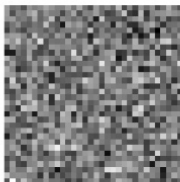
t = 200



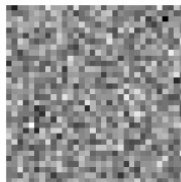
t = 300



t = 400



t = 500



Forward Process (Diffusion)

Reparameterization Trick:

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$$

$$q(x_{t-1} \mid x_{t-2}) = \mathcal{N}(x_{t-1}; \sqrt{1 - \beta_{t-1}} x_{t-2}, \beta_{t-1} I)$$

$$q(x_t \mid x_{t-2}) = \mathcal{N}(x_t; \sqrt{(1 - \beta_t)(1 - \beta_{t-1})} x_{t-2}, [(1 - \beta_t)\beta_{t-1} + \beta_t] I)$$

$\vdots$

$$q(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I)$$

$$\text{where } \bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s), \quad \alpha_t = 1 - \beta_t, \quad \text{and } \beta_t \in (0, 1), \quad t = 1, 2, \dots, T$$

x_t is the noisy data at time step t , and α_t controls the amount of noise added.

Forward Process (Diffusion)

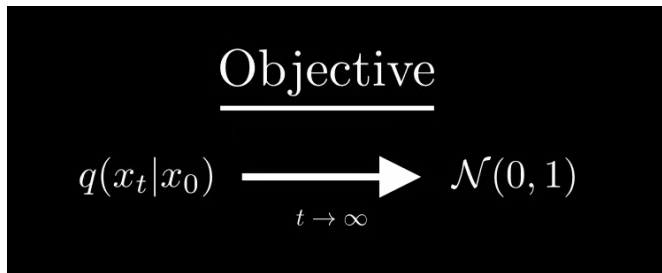


Figure: Forward Process.

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I) \quad (3)$$

- As $t \rightarrow \infty$, $\bar{\alpha}_t \rightarrow 0$, because $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$,
- x_t becomes increasingly noisy until it approximates a Gaussian distribution $\mathcal{N}(0, I)$.

Reverse Process (Denoising)

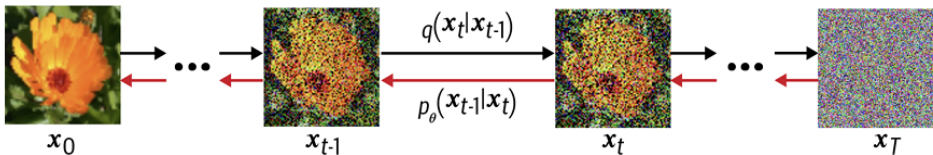


Figure: Reverse Process.

- Reconstructs the original data by denoising the noisy data in a series of steps reversing the forward diffusion.
- The reverse diffusion process is similar to the decoder in a VAE.
- In a VAE, the forward process (encoding images into noise) is learned / parameterized.
- In a Diffusion Model, the forward process is fixed / unparameterized.

Reverse Process (Denoising)

Modeled using a neural network to approximate the conditional probability $p_\theta(x_{t-1} \mid x_t)$

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}\left(\mathbf{x}_{t-1}; \boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0), \sigma_q^2(t)\mathbf{I}\right), \quad (4)$$

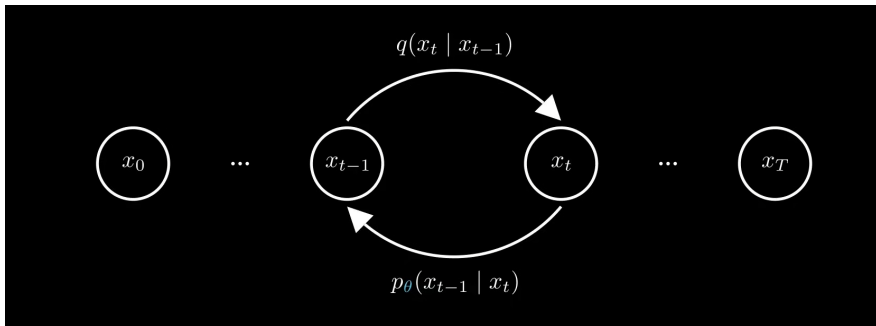


Figure: Reverse Process.

Reverse Process (Denoising)

(Proof in another slide)

where

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\alpha_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t$$

and

$$\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$$

We know from the forward process:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

So,

$$x_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} (x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon)$$

Substituting, we get:

$$\tilde{\mu}_t(x_t, \epsilon) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right). \quad (5)$$

Training Diffusion Models

Objective: We minimize the negative log-likelihood $-\log p_\theta(x_0)$.

Minimizing the objective function is equivalent to minimizing

$$\text{KL}(q(x_{t-1} \mid x_t, x_0) \parallel p_\theta(x_{t-1} \mid x_t))$$

so minimizing it gives $p_\theta(x_{t-1} \mid x_t) \approx q(x_{t-1} \mid x_t, x_0)$.

Algorithm 1 Training

```
1: repeat  
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$   
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$   
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
5:   Take gradient descent step on  
        $\nabla_\theta \left\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2$   
6: until converged
```

Figure: Training Algorithm.

Sampling from Diffusion Models

Algorithm 2 Sampling

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$ 
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
5: end for
6: return  $\mathbf{x}_0$ 
```

Figure: Sampling Algorithm.

Conditional Diffusion Models

- Extend standard diffusion models to **condition on extra information** y (e.g., class label, text, image).
- Goal: model $p_\theta(x_0 \mid y)$ instead of unconditional $p_\theta(x_0)$.
- Training uses **same denoising objective** as unconditional DDPM:

$$\mathbb{E}_{x_0, t, \epsilon} \left[\|\epsilon - \epsilon_\theta(x_t, t, y)\|^2 \right]. \quad (6)$$

- During sampling, model generates data consistent with condition y .
- Widely used in **text-to-image, class-conditional image generation, super-resolution, and inpainting**.
- Provides **fine-grained control** over generation while maintaining high sample quality.

Popular Applications of Diffusion Models

Text-to-Image Generation

- DALL-E 2 (OpenAI)
- Imagen (Google)

Diffusion Models Outperform GANs and VAEs

- **Stable Training:** No adversarial loss; avoids mode collapse and training instability common in GANs.
- **High-Quality & Diverse Samples:** VAE does one-shot generation whereas diffusion models iteratively denoise over T timesteps..
- **Exact Likelihood Estimation:** Allows likelihood-based training, enabling better probabilistic modeling and principled evaluation.

Posterior of Diffusion Models I

We start from the posterior:

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{x}_0) q(\mathbf{x}_{t-1} \mid \mathbf{x}_0)}{q(\mathbf{x}_t \mid \mathbf{x}_0)}. \quad (7)$$

By the Markov property of the forward process:

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{x}_0) = q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \alpha_t \mathbf{x}_{t-1}, (1 - \alpha_t) \mathbf{I}). \quad (8)$$

Using Markov and Reparameterization trick for $\mathbf{x}_t$ and $\mathbf{x}_{t-1}$ respectively:

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \frac{\mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t} \mathbf{x}_{t-1}, (1 - \alpha_t) \mathbf{I}) \mathcal{N}(\mathbf{x}_{t-1}; \sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0, (1 - \bar{\alpha}_{t-1}) \mathbf{I})}{\mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I})}. \quad (9)$$

From the definition ignoring the normalizing constants, we see that this expression has the form

Posterior of Diffusion Models II

$$\begin{aligned} \exp \bigg(& -\frac{1}{2} \frac{(\mathbf{x}_t - \sqrt{\alpha_t} \mathbf{x}_{t-1})^\top (\mathbf{x}_t - \sqrt{\alpha_t} \mathbf{x}_{t-1})}{1 - \alpha_t} \\ & -\frac{1}{2} \frac{(\mathbf{x}_{t-1} - \sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0)^\top (\mathbf{x}_{t-1} - \sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0)}{1 - \bar{\alpha}_{t-1}} \\ & +\frac{1}{2} \frac{(\mathbf{x}_t - \sqrt{\bar{\alpha}_t} \mathbf{x}_0)^\top (\mathbf{x}_t - \sqrt{\bar{\alpha}_t} \mathbf{x}_0)}{1 - \bar{\alpha}_t} \bigg). \end{aligned} \tag{10}$$

We will show that this expression matches

$$\mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0), \sigma_q^2(t) \mathbf{I}), \tag{11}$$

Posterior of Diffusion Models III

We can find $\sigma_q^2(t)$ by considering the coefficient of $-x_{t-1}^T x_{t-1}$ in the exponent of (13). This coefficient is given by

$$\frac{1}{2} \frac{\alpha_t}{1 - \alpha_t} + \frac{1}{2} \frac{1}{1 - \bar{\alpha}_{t-1}} = \frac{1}{2} \frac{\alpha_t(1 - \bar{\alpha}_{t-1}) + 1 - \alpha_t}{(1 - \alpha_t)(1 - \bar{\alpha}_{t-1})} = \frac{1}{2} \left(\frac{1 - \bar{\alpha}_t}{(1 - \alpha_t)(1 - \bar{\alpha}_{t-1})} \right),$$

where we used $\alpha_t \bar{\alpha}_{t-1} = \bar{\alpha}_t$. Hence,

$$\sigma_q^2(t) = \frac{(1 - \alpha_t)(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}. \quad (12)$$

Using the functional form of multivariate normal distribution, we can find $\mu_q(\mathbf{x}_t, \mathbf{x}_0)$ by considering the vector, say $\mathbf{v}$, such that $\mathbf{x}_{t-1}^\top \mathbf{v}$ is the cross-product in the exponent of (11). We see that

$$\frac{\mu_q(\mathbf{x}_t, \mathbf{x}_0)}{\sigma_q^2(t)} = \mathbf{v} = \frac{\sqrt{\alpha_t} \mathbf{x}_t}{1 - \alpha_t} + \frac{\sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0}{1 - \bar{\alpha}_{t-1}}. \quad (13)$$

Posterior of Diffusion Models IV

Hence, using (13),

$$\mu_q(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})\mathbf{x}_t + \sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)\mathbf{x}_0}{1 - \bar{\alpha}_t}. \quad (13)$$

We wish to compute a sample from the distribution in (12). This will allow us to perform the required transition along the backwards process. Our approach is to estimate the mean in (12) and then shift with an appropriate Gaussian in order to match the required variance.

If we know $\mathbf{x}_t$ and ϵ in (8), then we may write

$$\mathbf{x}_0 = \frac{\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \epsilon}{\sqrt{\bar{\alpha}_t}}. \quad (14)$$

Substituting this expression for $\mathbf{x}_0$ into (16), we see that the mean of $\mathbf{x}_{t-1}$, given $\mathbf{x}_t$ and $\mathbf{x}_0$, takes the form

Posterior of Diffusion Models V

$$\mu_q(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)}{1 - \bar{\alpha}_t \sqrt{\bar{\alpha}_t}} (\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \epsilon)$$

Substituting $\mathbf{x}_0 = (\mathbf{x}_t - \bar{\alpha}_t \epsilon) / \bar{\alpha}_t$:

$$\mu_q(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\alpha_t} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right). \quad (15)$$

Here, ϵ is the noise that drove the transition from $\mathbf{x}_0$ to $\mathbf{x}_t$. We train a neural network to predict ϵ given t and $\mathbf{x}_t$, which can then be used to estimate the mean for the backward process.